Programing Technique II

LECTURE 1

Course Objectives

- To provide students with general idea about the computer programming.
- To give a background about programming languages evolution.
- To introduce **C++** programming language and explain how practically it can be used (computer lab).

Course Assessment

Class Work	20 Marks.
Mid Exam	20 Marks.
Final Exam	60 Marks.

Introduction to C++ concepts

Computer Programs

It is also known as “Code”, it is a collection of instructions that can be executed to perform specific task.

Program/Code Components

Each program/code can be composed of

- Constants
- Variables
- Methods

Constants

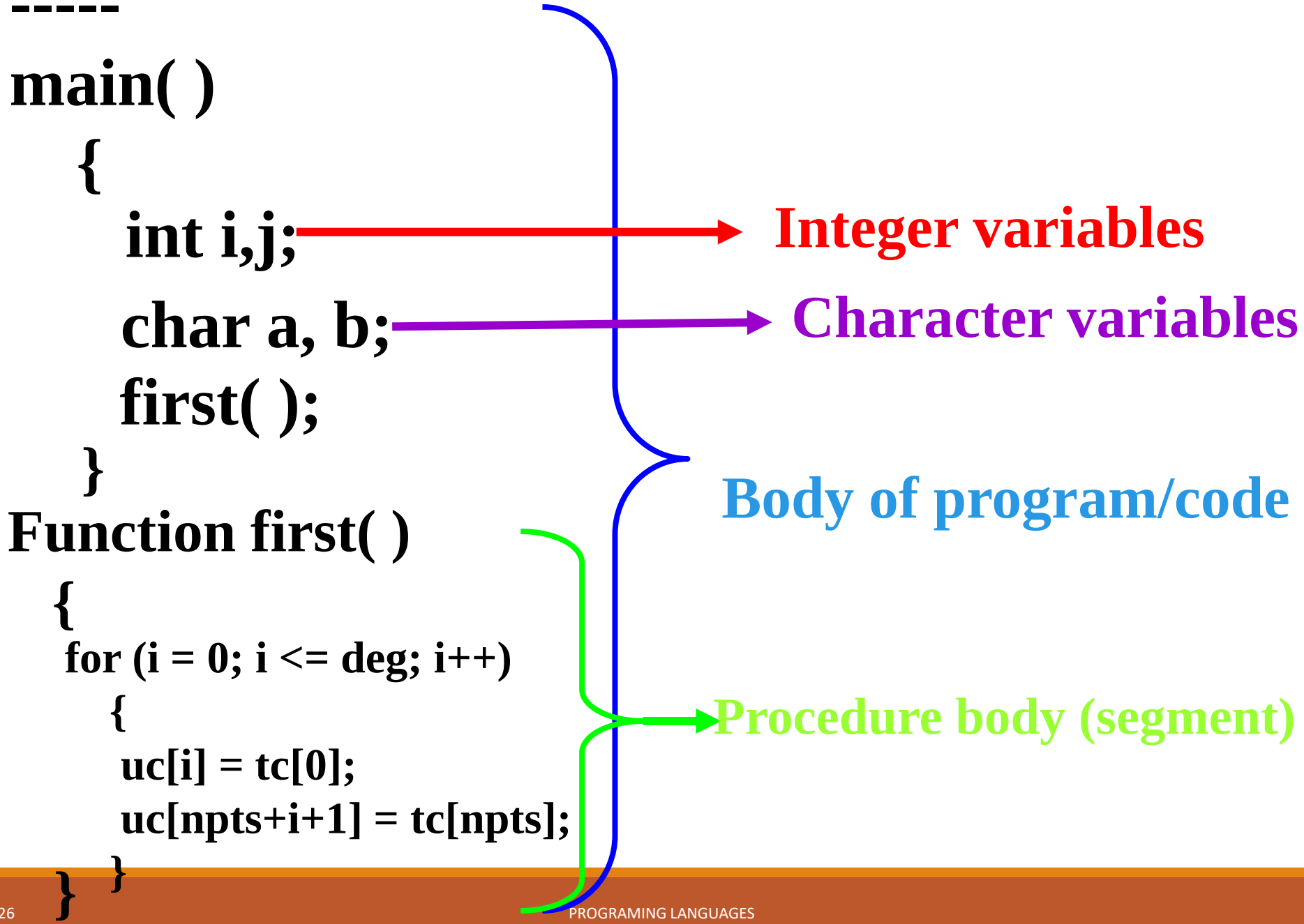
- Used to represented fixed values/data in the program body. (error C3892: 'variable_name' : you cannot assign to a variable that is const)

Variables

- Used to represented changeable values/data in the program

Methods

- Defined as procedures/functions, which considered as program segment that used to perform certain tasks of the whole program's tasks.


---
main( )
{
int i,j;
char a, b;
first( );
}
Function first( )
{
for (i = 0; i <= deg; i++)
{
uc[i] = tc[0];
uc[npts+i+1] = tc[npts];
}
}
Integer variables
Character variables
Body of program/code
Procedure body (segment)

C++ programming Language

Components of C++ Language:

- C++ program consists of statements, any statement consists of words, any words composes of a set of letters. So what are these letters:
  - Numbers: 0, 1, …, 9
  - Alphabet: A, B, …, Z, a, b, …, z
  - Symbols, Special characters, and mathematical operators: ($, { . ; / + - , others
- So words (names) in C++ can be built of alphabet, numbers, the symbols and _ (underscore).

Rules for C++ Names

names are **case-sensitive**
(age and AGE are
two different names)

1. A **name** must **start** with a **letter** or the **underscore** character
2. A name ***cannot start*** with a number
3. A name **can only contain alpha-numeric** characters and **underscores** (A-z, 0-9, and _)

❖**Examples**

- Valid names: count, sutd_name, age1, pl
- Invalid names: cnt*, stud name, 1age

Rules for C++ Names

- There are some words (names) not allowed to be used as user defined names. These word known **reserved words (keyword)** such as class, main, public, void, static, and others.

C++ Data Types

- Variables can store data of different types, and different data types can do different things.

- **C++ supports the following data types:**
  - Character.
  - Number.
    - Integer.
    - Real.
  - Boolean.
  - String.

Character Type (char)

Defines variable or constant of type character, that contains one character written as: ‘a’, ‘;’, ‘1’, …

the length of the variable/constant of type character in memory is 2 bytes

Examples:

- Char Code;
- Char Code = 'S';

Integer Type

defines variable or constant of with integer number such as 1, 30, 100, and so on. it is classified into 4 types :

Type	Length in Memory
byte	1 byte
short	2 byte
int	4 byte
Long	4 byte

Rules for integers:

- An integer must have at least one digit

- An integer must not have a decimal point

- An integer can be either positive or negative

Examples

```
byte count;

short count;

int count;

long count;
```

Real Type

defines variable or constant of with real number such as 1.0, 30.25, 100.75, and so on. It is classified into 2 types :

Type	Length in Memory
float	4 byte
double	8 byte

Rules for Reals:

- A Real must have at least one digit
- A Real may have a decimal point or not
- A Real can be either positive or negative

Examples

```
float count;

double count;
```

Boolean Type (bool):

defines variable or constant of type Boolean, which contains one of the two values: **true or false**

Examples

```
bool single;

bool single = true;
```

String Type (string):

defines variable or constant of type string, which contains a set of characters written as "Sudan", "Yes, welcome"

Examples

```
string name;

string name= "Ahmed";
```

Declaring variables in C++

Variables in C++ program can be declared (defined) as:

- Initially, write down the data type
- Then, write down a valid variable name

Examples

```
char  NN;       int var;       long var;     short var;

float var;    bool var;

string var;
```

Declaring Constants in C++

Constants in C++ program can be declared (defined) as:

- Initially, write down **(const)** keyword.
- Then, write down the data type
- last write down a valid variable name

Examples

const char var; **const** int var; **const** long var;

const short var; **const** float var; **const** bool var; **const** string var;

END

Introduction to C++ concepts

Lecture 2

Computer Programs

- It is also known as “Code”, it is a collection of instructions that can be executed to perform specific task.
- Program/Code Components
- Each program/code can be composed of
  - Constants
  - Variables
  - Methods

Constants

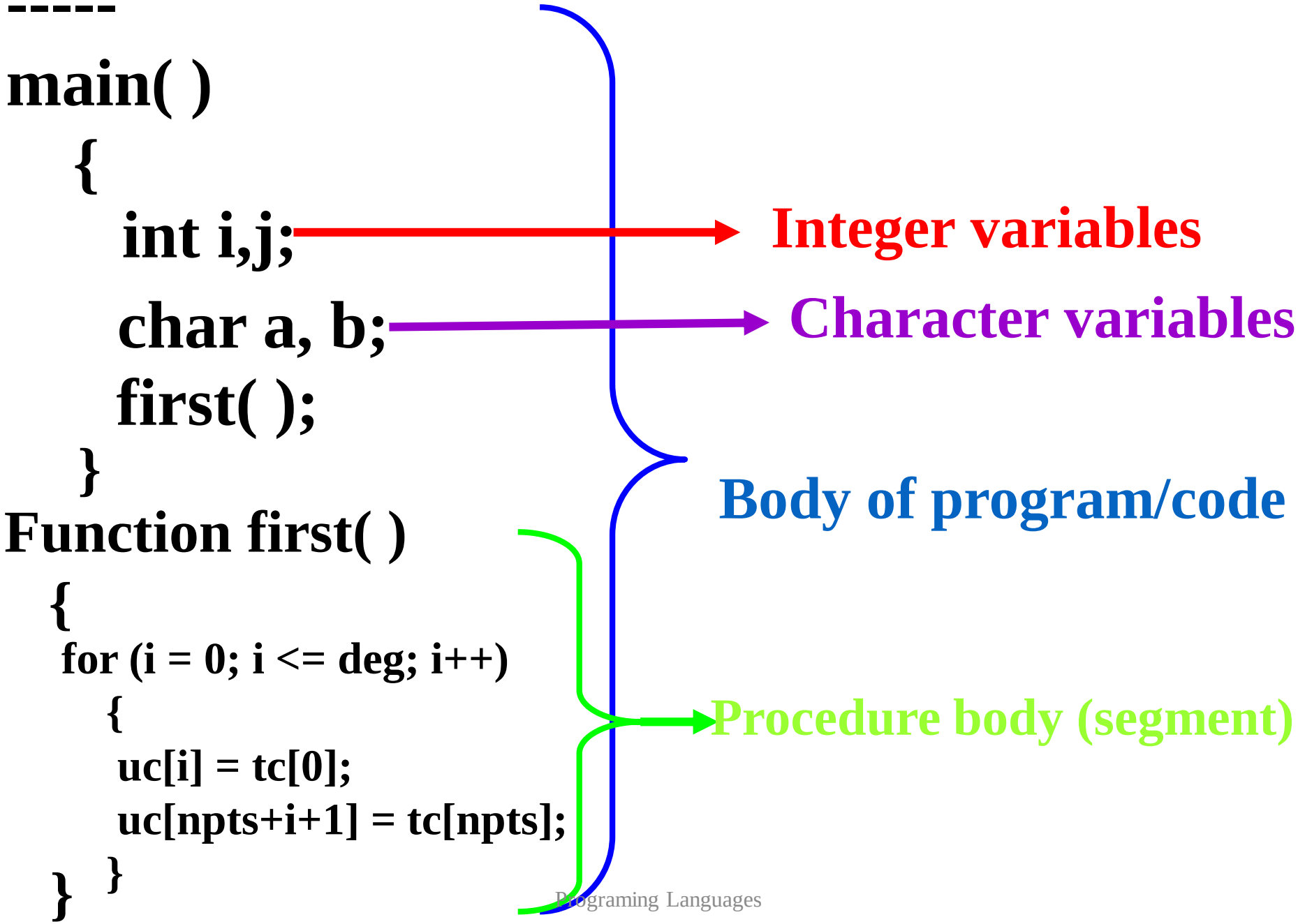
- Used to represented fixed values/data in the program body. (error C3892: 'variable_name' : you cannot assign to a variable that is const)

Variables

- Used to represented changeable values/data in the program

Methods

- Defined as procedures/functions, which considered as program segment that used to perform certain tasks of the whole program’s tasks.


-----
main( )
{
int i,j;
char a, b;
first( );
}
Function first( )
{
for (i = 0; i <= deg; i++)
{
uc[i] = tc[0];
uc[npts+i+1] = tc[npts];
}
}
Integer variables
Character variables
Body of program/code
Procedure body (segment)

C++ programming Language

- Components of C++ Language:
  - C++ program consists of statements, any statement consists of words, any words composes of a set of letters. So what are these letters:
    - Numbers: 0, 1, …, 9
    - Alphabet: A, B, …, Z, a, b, …, z
    - Symbols, Special characters, and mathematical operators: ($, { . ; / + - , others
  - So words (names) in C++ can be built of alphabet, numbers, the symbols and _ (underscore).

Rules for C++ Names

names are **case-sensitive**
(age and AGE are
two different names)

1. A **name** must **start** with a **letter** or the **underscore** character
2. A name ***cannot start*** with a number
3. A name **can only contain** **alpha-numeric** characters and **underscores** (A-z, 0-9, and _)

❖**Examples**

- Valid names: count, sutd_name, age1, pl
- Invalid names: cnt*, stud name, 1age

Rules for C++ Names

❖There are some words (names) not allowed to be used as user defined names. These word known **reserved words (keyword)** such as class, main, public, void, static, and others.

C++ Data Types

❖Variables can store data of different types, and different data types can do different things.

❖**C++ supports the following data types:**

- Character.
- Number.
  - Integer.
  - Real.
- Boolean.
- String.

Character Type (char)

- Defines variable or constant of type character, that contains one character written as: ‘a’, ‘;’, ‘1’, …
- the length of the variable/constant of type character in memory is 2 bytes
- **Examples**:
  - Char Code;
  - Char Code = 'S';

Integer Type

- defines variable or constant of with integer number such as 1, 30, 100, and so on. it is classified into 4 types :

Type	Length in Memory
byte	1 byte
short	2 byte
int	4 byte
Long	4 byte

Rules for integers:

- An integer must have at least one digit
- An integer must not have a decimal point
- An integer can be either positive or negative

Examples

```
byte count;

short count;

int count;

long count;
```

Real Type

- defines variable or constant of with real number such as 1.0, 30.25, 100.75, and so on. It is classified into 2 types :

Type	Length in Memory
float	4 byte
double	8 byte

Rules for Reals:

- A Real must have at least one digit
- A Real may have a decimal point or not
- A Real can be either positive or negative

Examples

```
float count;

double count;
```

Boolean Type (bool):

- defines variable or constant of type Boolean, which contains one of the two values: **true or false**

Examples

```
bool single;

bool single = true;
```

String Type (string):

- defines variable or constant of type string, which contains a set of characters written as "Sudan", "Yes, welcome"

Examples

```
string name;

string name= "Ahmed";
```

Declaring variables in C++

- Variables in C++ program can be declared (defined) as:
  - Initially, write down the data type
  - Then, write down a valid variable name

Examples

```
char  var;
int var;
long var;
short var;
float var;
 bool var;
string var;
```

Declaring Constants in C++

- Constants in C++ program can be declared (defined) as:
  - Initially, write down **(const)** keyword.
  - Then, write down the data type
  - last write down a valid variable name

Examples

const char var;

const int var;

const long var;

const short var;

const float var;

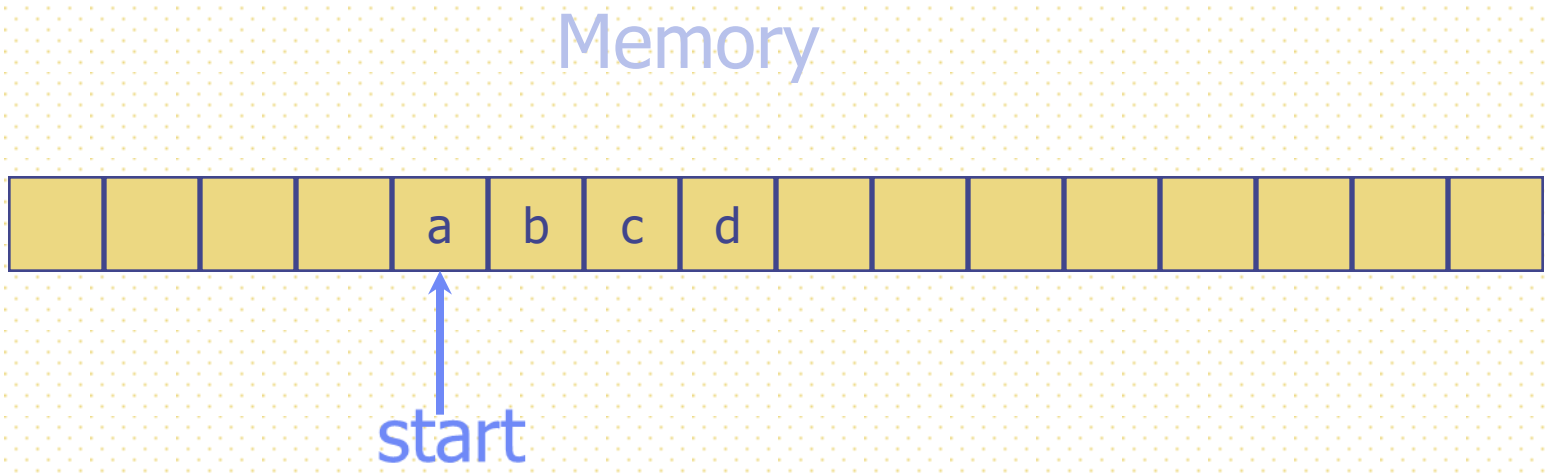
const bool var;

const string var;

Arrays

Lecture#3

Array



- Storing data in a sequential memory locations
- Access each element using integer index
- Very basic, popular, and simple

- 1-dimensional array.

Currency Last Trade	U.S. $ N/A	Aust $ Oct 14	U.K. £ Oct 14	Can $ Oct 14	DMark Oct 14	FFranc Oct 14	¥en Oct 14	SFranc Oct 14	Euro 12:39AM
U.S. $	1	0.6493	1.663	0.675	0.5513	0.1644	0.009316	0.6784	1.082

- 3-dimensional array (3rd dimension is the day).

Currency Last Trade	U.S. $ N/A	Aust $ Oct 14	U.K. £ Oct 14	Can $ Oct 14	DMark Oct 14	FFranc Oct 14	¥en Oct 14	SFranc Oct 14	Euro 12:39AM
U.S. $	1	0.6493	1.663	0.675	0.5513	0.1644	0.009316	0.6784	1.082
Aust $	1.54	1	2.562	1.04	0.8491	0.2532	0.01435	1.045	1.666
U.K. £	0.6012	0.3904	1	0.4058	0.3314	0.09883	0.005601	0.4079	0.6505
Can $	1.481	0.9619	2.464	1	0.8167	0.2435	0.0138	1.005	1.603
DMark	1.814	1.178	3.017	1.224	1	0.2982	0.0169	1.231	1.963
FFranc	6.083	3.95	10.12	4.106	3.354	1	0.05667	4.127	6.582
¥en	107.3	69.7	178.5	72.46	59.18	17.64	1	72.82	116.1
SFranc	1.474	0.9571	2.452	0.995	0.8126	0.2423	0.01373	1	1.595
Euro	0.9242	0.6001	1.537	0.6238	0.5095	0.1519	0.00861	0.627	1

Array Applications

- Given a list of test scores, determine the maximum and minimum scores.
- Read in a list of student names and rearrange them in alphabetical order (sorting).
- Given the height measurements of students in a class, output the names of those students who are taller than average.

Array Declaration

- Syntax:

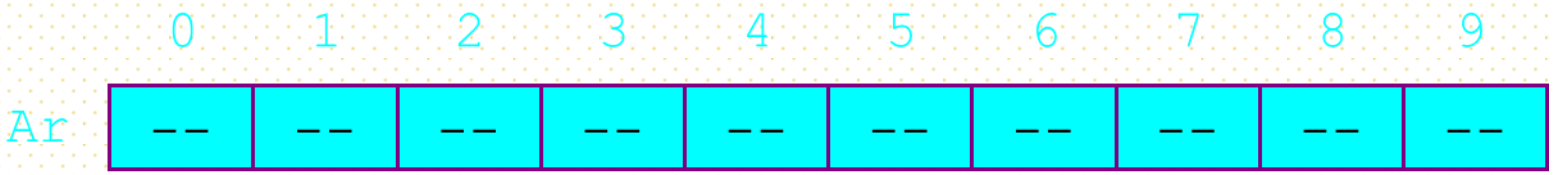
  <type> <arrayName>[<array_size>]

  Ex. int Ar[10];

- The array elements are all values of the type **<type>.**
- The size of the array is indicated by **<array_size>**, the number of elements in the array.
- **<array_size>** must be an **int** constant or a constant expression. Note that an array can have multiple dimensions.

Array Declaration

```
// array of 10 uninitialized ints
int Ar[10];
```



Subscripting

- Declare an array of 10 integers:

```
int Ar[10];     // array of 10 ints
```

- To access an individual element we must apply a subscript to array named **`Ar`**.
  - ✓ A subscript is a bracketed expression.
    - The expression in the brackets is known as the index.
  - ✓ First element of array has index 0.

    `Ar[0]`
  - ✓ Second element of array has index 1, and so on.

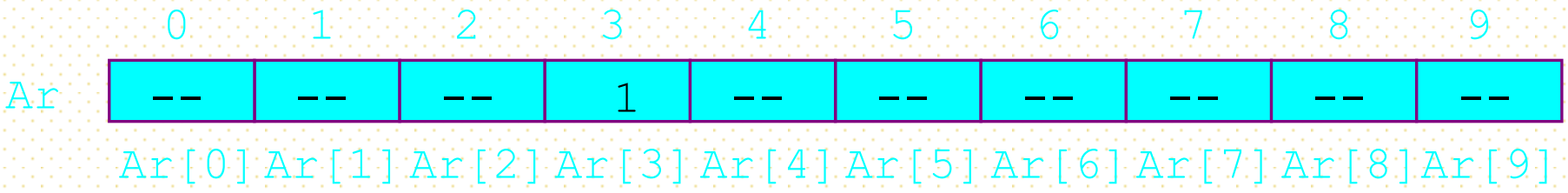
    `Ar[1], Ar[2], Ar[3],…`
  - ✓ Last element has an index one less than the size of the array.

    `Ar[9]`
- Incorrect indexing is a common error.

Subscripting

```
// array of 10 uninitialized ints
int Ar[10];

Ar[3] = 1;
int x = Ar[3];
```



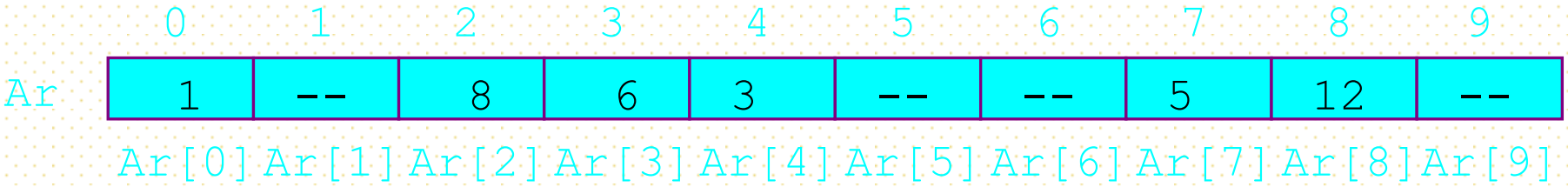
Subscripting Example 1

```
   //For loop to fill & print a 10-int array
#include <iostream>
using namespace std;
   int main ( ) {
      int index, ar[10];  // array for 10 integers
      // Read in 10 elements.
      cout << "Enter 10 integers: ";
      for(index = 0; index < 10; index ++)
            cin >> ar[index];
      cout << endl;
      cout << "The integers are ";
      for(index = 0; index < 10; index ++)
            cout << ar[index] << " ";
      cout << endl;
      return 0;
   }
```

Array Element Manipulation Ex. 3

- Consider

```
int Ar[10], i = 7, j = 2, k = 4;
Ar[0] = 1;
Ar[i] = 5;
Ar[j] = Ar[i] + 3;
Ar[j+1] = Ar[i] + Ar[0];
Ar[Ar[j]] = 12;
cin >> Ar[k]; // where the next input value is 3
```

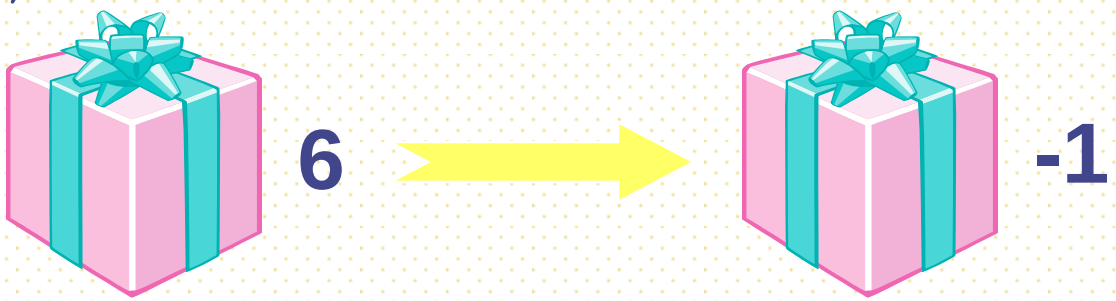


Array Initialization Ex. 4

```
int Ar[10] = {9, 8, 7, 6, 5, 4, 3, 2, 1, 0};
```

	0	1	2	3	4	5	6	7	8	9
Ar	9	8	7	6	5	4	3	2	1	0

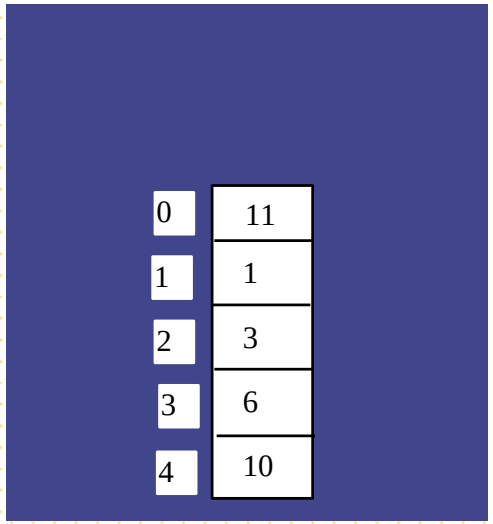
```
Ar[3] = -1;
```



	0	1	2	3	4	5	6	7	8	9
Ar	9	8	7	-1	5	4	3	2	1	0

Program with Arrays

```
int main()
{
   int values[5]= {11,1,3,6,10};
   for (int i = 1; i < 5; i++)
  {
     values[i] = values[i] + values[i-1];
   }
   values[0] = values[1] + values[4];
}
```



Printing arrays

To print an array, you have to print each element in the array using a loop like the following:

```
for (int i = 0; i < ARRAY_SIZE; i++)
{
  cout << myList[i] << " ";
}
```

Copying Arrays

Can you copy array using a syntax like this?

list = myList;

This is not allowed in C++. You have to copy individual elements from one array to the other as follows:

```
for (int i = 0; i < ARRAY_SIZE; i++)
{
  list[i] = myList[i];
}
```

Summing All Elements

Use a variable named total to store the sum. Initially total is 0. Add each element in the array to total using a loop like this:

```
double total = 0;
for (int i = 0; i < ARRAY_SIZE; i++)
{
  total += myList[i];
}
```

Finding the Largest Element

Use a variable named max to store the largest element. Initially max is myList[0]. To find the largest element in the array myList, compare each element in myList with max, update max if the element is greater than max.

```
double max = myList[0];
for (int i = 1; i < ARRAY_SIZE; i++)
{
  if (myList[i] > max) max = myList[i];
}
```

Shifting Elements

```
double temp = myList[0]; // Retain the first element
// Shift elements left
for (int i = 1; i < myList.length; i++)
{
  myList[i - 1] = myList[i];
}
// Move the first element to fill in the last position
myList[myList.length - 1] = temp;
```

Questions?

Lecture #4
Pointers and References

PROGRAMMING FUNDAMENTALS

Introduction

- Programming Fundamentals
- Topic: Pointers and References
- Level: First Year Students

Learning Outcomes

- Understand memory addresses
- Define and declare pointers
- Use address operator (&)
- Use dereference operator (*)
- Understand arrays and pointers
- Use pointers with functions

Review: Variables in Memory

- Every variable has a value and a memory address.
- Example: int x = 10;
- Address | Value
- 1000 | 10

What is a Pointer?

- A pointer is a variable that stores the memory address of another variable.
- Code:
- int x = 10;
- int *p = &x;

Pointer Syntax

- int *p;
- float *f;
- char *c;

Address Operator (&)

- The & operator returns the address of a variable.
- Code:
- int x = 10;
- cout << &x;
- Possible Output: 0x61ff08

Example: Storing Address in Pointer

```
#include <iostream>
using namespace std;
int main() {
 int x = 10;
 int *p = &x;
 cout << "Value of x = " << x << endl;
 cout << "Address of x = " << &x << endl;
 cout << "Pointer p stores = " << p << endl;
}
```

Output:

Value of x = 10

Address of x = 0x61ff08

Pointer p stores = 0x61ff08

Dereference Operator (*)

- The * operator accesses the value stored at an address.
- Code:
- int x = 10;
- int *p = &x;
- cout << *p;
- Output: 10

p vs *p

- p -> Address stored in pointer
- *p -> Value stored at that address

Memory Illustration

- Variable x
- Address 1000 -> Value 10
- Pointer p
- Address 2000 -> Value 1000

Modifying Variables Using Pointers

- Code:
- int x = 10;
- int *p = &x;
- *p = 25;
- cout << x;
- Output: 25

Pointers and Arrays

- Array name represents the address of the first element.
- Example:
- int arr[3] = {10,20,30};

Array and Pointer Example

- cout << arr << endl;
- cout << &arr[0] << endl;
- Output: Same address

Pointer Arithmetic

- int arr[3] = {10,20,30};
- int *p = arr;
- *p = 10
- *(p+1) = 20
- *(p+2) = 30

Pointer Arithmetic Example

- Code outputs:
- 10
- 20
- 30

Passing by Address

- Pointers allow functions to modify original variables.
- void changeValue(int *p) {
- *p = 50;
- }

Complete Example

- changeValue(&x);
- cout << x;
- Output: 50

Advantages of Pointers

- Dynamic memory allocation
- Efficient data passing
- Working with arrays and strings
- Building data structures

Common Mistakes

- Using uninitialized pointers
- Forgetting the address operator (&)

Null Pointer

- int *p = nullptr;
- Used to avoid invalid memory access.

Real-Life Analogy

- Variable value = House
- Memory address = House location
- Pointer = Paper containing the address

Practice Question 1

- int x = 5;
- int *p = &x;
- cout << *p;
- Output: 5

Practice Question 2

- int x = 10;
- int *p = &x;
- *p = 30;
- cout << x;
- Output: 30

Practice Question 3

- int x = 20;
- int *p = _____;
- Answer: &x

Summary

- Pointers store memory addresses
- & gets address
- * accesses value
- Pointers work with arrays and functions

Homework

- Create a pointer program
- Print values using pointer arithmetic
- Write swap function using pointers

Thank You

- Questions?